

DockLens

Scanner de Securite pour Images Docker

Documentation Technique — BOUARABA Nassima, FRESSE Balkis — 4SI A

DockLens est un outil DevSecOps developpe en Python qui analyse des images Docker pour detecter des vulnerabilites connues, sans jamais executer les conteneurs. Il combine deux sources : l'API OSV (Google) et l'API EUVD (Union Europeenne), avec cache SQLite local, export PDF/JSON et API REST FastAPI.

1. Architecture du projet

```
Docklens/
├── main.py                # Point d'entree CLI
├── cli/
│   ├── menu.py           # Commandes Typer (scan, --export, --no-cache)
│   ├── output.py         # Affichage Rich + export PDF/JSON
│   └── scanner/
│       ├── docker_image.py # Validation et pull de l'image
│       ├── extractor.py    # Extraction du filesystem (layers TAR)
│       └── packages.py     # Lecture paquets APK/DPKG + version Alpine
├── vulns/
│   ├── osv_client.py      # Client API OSV (Google)
│   ├── euvd_client.py    # Client API EUVD (Europe/ENISA)
│   └── scanner.py        # Orchestration + cache + tri par severite
├── cache/
│   ├── models.py         # Modele SQLAlchemy (CachedVulnerability)
│   ├── repository.py     # Cache SQLite avec TTL 24h
│   └── migrations/       # Migrations Alembic
├── cve/
│   └── cpe.py            # Generation identifiants CPE 2.3
├── api/
│   └── main.py           # API REST FastAPI (POST /scan, GET /health)
├── config/
│   └── settings.py       # Configuration (chemin cache, settings)
└── tests/
    ├── test_docker_image.py
    ├── test_extractor.py
    ├── test_packages.py
    └── unit/
        ├── test_cache_repository.py
        └── test_cpe.py
```

2. Flux d'execution

```
python main.py scan alpine:3.18 [--export rapport.pdf] [--no-cache]
```

```
|
v
```

```
[1] Valider/puller l'image Docker (docker SDK)
```

```
|
v
```

```
[2] docker save -> image.tar (filesystem complet en TAR)
    |
    v
[3] Lire manifest.json -> ordre exact des layers
    |
    v
[4] Extraire les layers TAR -> /tmp/docklens_xxx/filesystem/
    |
    v
[5] Lire /etc/alpine-release -> "Alpine:v3.18"
    Detecter distro: Alpine (/lib/apk/db/installed)
                  ou Debian (/var/lib/dpkg/status)
    |
    v
[6] Extraire liste des paquets {name, version, ecosystem}
    + generer identifiants CPE 2.3 (cve/cpe.py)
    |
    v
[7] Pour chaque paquet:
    -> Verifier cache SQLite (TTL 24h)
    -> Si absent: requete OSV API
    -> Stocker en cache
    |
    v
[8] Pour chaque CVE trouvee:
    -> Verifier cache SQLite
    -> Si absent: requete EUVD API (enrichissement europeen)
    -> Stocker en cache
    |
    v
[9] Trier par severite: CRITICAL > HIGH > MEDIUM > LOW > UNKNOWN > NONE
    |
    v
[10] Afficher tableau Rich dans le terminal
    + exporter PDF ou JSON si --export specifie
```

3. Module scanner/

3.1 docker_image.py — Validation de l'image

```
def get_docker_client():
    try:
        client = docker.from_env()
        client.ping()          # teste la connexion au daemon Docker
        return client
    except DockerException:
        print("[ERROR] Docker is not running.")
        sys.exit(1)

def validate_and_pull_image(image_name: str):
    client = get_docker_client()
    try:
        image = client.images.get(image_name)  # cherche en local
        print(f"[OK] Image found locally: {image_name}")
        return image
    except ImageNotFound:
        image = client.images.pull(image_name) # telecharge depuis Hub
        return image
```

3.2 extractor.py — Extraction du filesystem

Exporte l'image en TAR via docker save, lit manifest.json pour l'ordre des layers, extrait chaque layer. Les fichiers .wh. (whiteout) sont ignores car ils marquent des suppressions de fichiers entre layers.

```
def extract_filesystem(image_name: str) -> Path:
    image = client.images.get(image_name)
    # docker save -> ecriture du TAR
    with open(image_tar, "wb") as f:
        for chunk in image.save(named=True):
            f.write(chunk)

    with tarfile.open(image_tar, "r") as tar:
        manifest = json.load(tar.extractfile("manifest.json"))
        layers = manifest[0]["Layers"]  # ordre exact des layers

        for layer_path in layers:
            layer_file = tar.extractfile(layer_path)
            with tarfile.open(fileobj=layer_file) as layer_tar:
                for member in layer_tar.getmembers():
                    if ".wh." in member.name:  # whiteout = fichier supprime
                        continue
                    layer_tar.extract(member, path=filesystem_dir, filter="data")
```

3.3 packages.py — Extraction des paquets + version Alpine

```
def get_alpine_version(fs_path: str) -> str:
    # /etc/alpine-release contient "3.18.12"
    # On construit "Alpine:v3.18" pour OSV
    with open(os.path.join(fs_path, "etc", "alpine-release")) as f:
        version = f.read().strip()  # "3.18.12"
    parts = version.split(".")
```

```

    return f"Alpine:v{parts[0]}.{parts[1]}"    # "Alpine:v3.18"

def parse_apk_packages(fs_path: str) -> list:
    # /lib/apk/db/installed: blocs separees par lignes vides
    # P:busybox  <- nom      V:1.36.1-r7  <- version
    ecosystem = get_alpine_version(fs_path)
    for line in f:
        if line.startswith("P:"): current["name"] = line[2:].strip()
        elif line.startswith("V:"): current["version"] = line[2:].strip()
        elif line == "":
            current["ecosystem"] = ecosystem    # "Alpine:v3.18"
            packages.append(current); current = {}

def parse_dpkg_packages(fs_path: str) -> list:
    # /var/lib/dpkg/status: "Package: bash" / "Version: 5.1-2"
    for line in f:
        if line.startswith("Package:"): current["name"] = ...
        elif line.startswith("Version:"): current["version"] = ...
        elif line == "":
            current["ecosystem"] = "Debian"
            packages.append(current); current = {}

```

4. Module vulns/ — Detection des vulnerabilites

4.1 API OSV (Google Open Source Vulnerabilities)

API REST gratuite de Google. On envoie nom + version + ecosystem, elle retourne les CVE avec version corrigee et score CVSS.

```
# Requete POST vers OSV
POST https://api.osv.dev/v1/query
{
  "package": {"name": "busybox", "ecosystem": "Alpine:v3.18"},
  "version": "1.36.1-r7"
}

# Reponse OSV (simplifie)
{
  "vulns": [{
    "id": "ALPINE-CVE-2025-46394",
    "details": "In tar in BusyBox through 1.37.0...",
    "affected": [{"ranges": [{"events": [
      {"introduced": "0"}, {"fixed": "1.37.0-r27"}
    ]}]},
    "severity": [{"type": "CVSS_V3",
      "score": "CVSS:3.1/AV:L/AC:L/PR:L/UI:N/S:U/C:N/I:L/A:N"}]
  }]
}
```

4.2 Calcul de la severite CVSS

Le score CVSS brut est une chaine comme CVSS:3.1/AV:L/AC:L/C:N/I:N/A:H. On extrait C (Confidentiality), I (Integrity), A (Availability) pour calculer un score 0-10.

```
def parse_cvss_severity(score_string: str) -> str:
    impact_map = {"N": 0.0, "L": 0.22, "H": 0.56}
    parts = {}
    for part in score_string.split("/"):
        if ":" in part: key, val = part.split(":", 1); parts[key] = val

    c = impact_map.get(parts.get("C", "N"), 0.0) # Confidentiality
    i = impact_map.get(parts.get("I", "N"), 0.0) # Integrity
    a = impact_map.get(parts.get("A", "N"), 0.0) # Availability
    score = (c + i + a) * 10 / 1.68 # score 0-10

    if score >= 9.0: return "CRITICAL"
    elif score >= 7.0: return "HIGH"
    elif score >= 4.0: return "MEDIUM"
    elif score > 0.0: return "LOW"
    else: return "NONE"
```

4.3 API EUVD (European Union Vulnerability Database)

Base europeenne lancee par l'ENISA en avril 2025. Enrichit chaque CVE avec un identifiant europeen (EUVD-XXXX-XXXXX) et son propre score CVSS. IMPORTANT: la vraie URL API est <https://euvdservices.enisa.europa.eu> (pas le site web qui retourne du HTML).

```
# Requete GET vers EUVD
GET https://euvservices.enisa.europa.eu/api/search?text=CVE-2025-26519&page=0&size=5
Header: accept: application/json
```

```
# Reponse EUVD
```

```
{
  "items": [{
    "id": "EUVD-2025-4206",
    "baseScore": 8.1,
    "baseScoreVersion": "3.1",
    "aliases": "CVE-2025-26519
GHSA-xpv5-92cc-8f65
"
  }]
}
```

```
def query_euvsd(cve_id: str) -> dict:
    response = httpx.get(EUVD_API_URL,
        params={"text": cve_id, "page": 0, "size": 5},
        headers={"accept": "application/json"}, timeout=10)
    for item in response.json().get("items", []):
        if cve_id in item.get("aliases", ""):
            return {"euvsd_id": item["id"], "base_score": item["baseScore"],
                "url": f"https://euvsd.enisa.europa.eu/ENISA/{item['id']}"}
    return {}
```

5. Cache SQLite — Optimisation des requetes API

Pour eviter de re-interroger OSV et EUVD a chaque scan, DockLens stocke les resultats dans une base SQLite locale avec un TTL de 24h. Implementee avec SQLAlchemy + Alembic pour les migrations.

```
# cache/models.py – Table SQLite
class CachedVulnerability(Base):
    __tablename__ = "cached_vulnerabilities"
    id = Column(Integer, primary_key=True)
    package_key = Column(String) # ex: "Alpine:v3.18:busybox:1.36.1-r7"
    source = Column(String) # "osv" ou "euvd"
    payload = Column(Text) # JSON des resultats
    fetched_at = Column(DateTime)
    expires_at = Column(DateTime) # TTL 24h

# cache/repository.py – Utilisation
cache = CacheRepository(db_path=Path("~/cache/docklens/cache.db"))

result = cache.get("Alpine:v3.18:busybox:1.36.1-r7", "osv")
if result is None:
    result = query_osv("busybox", "1.36.1-r7", "Alpine:v3.18")
    cache.set("Alpine:v3.18:busybox:1.36.1-r7", "osv", result)

# Bypass du cache: python main.py scan alpine:3.18 --no-cache
```

6. Identifiants CPE 2.3 (cve/cpe.py)

CPE (Common Platform Enumeration) est un format standardise pour identifier precisement un logiciel. Chaque paquet extrait recoit son identifiant CPE.

```
# Format CPE 2.3:
# cpe:2.3:a:<vendor>:<product>:<version>:*:*:*:*:*:*

# Exemple pour busybox 1.36.1-r7:
# cpe:2.3:a:busybox:busybox:1.36.1-r7:*:*:*:*:*:*

def enrich_packages(packages: list) -> list:
    for pkg in packages:
        name, version = pkg["name"], pkg.get("version")
        if not version:
            cpe = None
        else:
            # cherche dans cpe_lookup.json sinon heuristique
            key = f"{ecosystem.lower()}:{name}"
            entry = LOOKUP_TABLE.get(key)
            vendor = entry["vendor"] if entry else name
            product = entry["product"] if entry else name
            cpe = f"cpe:2.3:a:{vendor}:{product}:{version}:*:*:*:*:*:*"
        pkg["cpe"] = cpe
    return packages
```

7. API REST FastAPI (api/main.py)

Expose DockLens comme un service HTTP. Permet a d'autres outils (CI/CD, scripts) d'utiliser DockLens sans passer par le terminal. Documentation automatique Swagger disponible sur <http://localhost:8000/docs>

```
# Lancer l'API
uvicorn api.main:app --reload --port 8000

# Endpoints disponibles:
GET /          -> statut de l'API
GET /health    -> {"status": "ok"}
POST /scan     -> scanner une image

# Exemple d'utilisation avec curl
curl -X POST "http://localhost:8000/scan" \
  -H "Content-Type: application/json" \
  -d '{"image": "python:3.9-alpine"}' | jq '.vulnerabilities_found'
# -> 6

# Filtrer seulement les CRITICAL
curl -X POST "http://localhost:8000/scan" \
  -H "Content-Type: application/json" \
  -d '{"image": "python:3.9-alpine"}' | jq '.findings[] | select(.severity == "CRITICAL")'

# Reponse JSON structure:
{
  "image": "python:3.9-alpine",
  "total_packages": 38,
  "vulnerabilities_found": 6,
  "findings": [
    {
      "package": "musl", "version": "1.2.5-r10",
      "severity": "CRITICAL", "cve": "ALPINE-CVE-2026-40200",
      "fix": "1.2.6-r2", "command": "apk add musl=1.2.6-r2",
      "euvd_id": "EUVD-2026-21496", "euvd_score": 8.1
    }
  ]
}
```

8. Export de rapports (--export)

DockLens peut generer des rapports exportables apres chaque scan. Utile pour documenter les resultats ou les partager.

```
# Exporter en PDF
python main.py scan python:3.9-alpine --export rapport.pdf

# Exporter en JSON
python main.py scan python:3.9-alpine --export rapport.json

# Les deux en meme temps: d'abord affiche le tableau, puis genere le fichier
python main.py scan debian:11 --export rapport_debian.pdf
```


9. Commandes importantes

```
# === SCAN CLI ===
source .venv/bin/activate # activer le venv
python main.py scan alpine:3.18 # scan tableau Rich
python main.py scan debian:11 # scan Debian/DPKG
python main.py scan python:3.9-alpine --format json # scan JSON
python main.py scan python:3.9-alpine --export rapport.pdf # export PDF
python main.py scan python:3.9-alpine --no-cache # ignorer le cache

# === API REST ===
uvicorn api.main:app --reload --port 8000 # lancer l'API
curl -X POST http://localhost:8000/scan \
  -H "Content-Type: application/json" \
  -d '{"image": "alpine:3.18"}' | jq .vulnerabilities_found

# === TESTS ===
python -m tests.test_output # test affichage
python -m tests.test_docker_image # test validation image
python -m tests.test_extractor # test extraction
python -m tests.test_packages # test paquets
python -m tests.unit.test_cache_repository # test cache SQLite
python -m tests.unit.test_cpe # test CPE

# === API OSV directement ===
python -c "
from vulns.osv_client import query_osv
results = query_osv('busybox', '1.36.1-r7', 'Alpine:v3.18')
for r in results: print(r['id'], r['severity'], r['fixed'])
"

# === API EUVD directement ===
python -c "
from vulns.euvd_client import query_euvd
print(query_euvd('CVE-2025-26519'))
"

# === GIT workflow ===
git checkout -b feat/ma-feature # nouvelle branche
git add fichier.py # ajouter fichier
git commit -m "feat: description" # commiter
git push origin feat/ma-feature # pousser
git pull origin main --rebase # recuperer main
```

10. Points clés pour la soutenance

Concept	Explication
Analyse statique	L'image n'est jamais lancée (pas de docker run). On utilise docker save pour exporter le filesystem en TAR et on l'analyse avec OSV.
Ecosystem OSV versionné	OSV exige 'Alpine:v3.18' et non 'Alpine'. On lit /etc/alpine-release dans le filesystem extrait pour obtenir la bonne version.
Double source de vulns	OSV (Google) trouve les CVE et la version corrigée. EUVD (ENISA/Europe) enrichit avec l'identifiant européen.
API EUVD	Le vrai endpoint est https://euvidservices.enisa.europa.eu/api/search . Le site web euvid.enisa.europa.eu retourne une page 404.

Layers Docker	Une image = plusieurs layers superposes. manifest.json donne l'ordre exact. Les fichiers .wh. (whiteout) mar
Cache SQLite	Les resultats OSV et EUVD sont caches 24h en SQLite pour eviter de re-interroger les API a chaque scan du
CPE 2.3	Format standard pour identifier un logiciel: cpe:2.3:a:vendor:product:version:*. Permet l'interopabilite avec c
API REST FastAPI	Expose DockLens via HTTP pour integration CI/CD. Documentation Swagger auto sur http://localhost:8000/d
Export PDF/JSON	Option --export pour generer des rapports persistants. Utile pour documenter les audits de securite.
Tri par severite	CRITICAL > HIGH > MEDIUM > LOW > UNKNOWN > NONE via un dictionnaire de priorites et un sort()).

Stack technique : Python 3.13 • Typer • Rich • httpx • docker SDK • SQLAlchemy • Alembic • FastAPI • ReportLab